

NorthwindCatalog

Week 15 Assessment – Project Report

Submitted by: Aaroh

Date: April 18, 2026

Technology: ASP.NET Core MVC | Entity Framework Core | AutoMapper | Repository Pattern

1. Project Initialization & DB First Approach

A blank solution named NorthwindCatalog was created with the following three projects:

- NorthwindCatalog.Services (Class Library)
- NorthwindCatalog.Web (ASP.NET Core MVC)
- NorthwindCatalog.Tests (xUnit Test Project)

The following NuGet packages were installed in the Services project:

- Microsoft.EntityFrameworkCore.SqlServer
- Microsoft.EntityFrameworkCore.Tools
- AutoMapper.Extensions.Microsoft.DependencyInjection

The database was scaffolded using the EF Core CLI command:

```
dotnet ef dbcontext scaffold "YourConnectionString"  
Microsoft.EntityFrameworkCore.SqlServer -o Models
```

Screenshot below shows the solution structure after project setup:

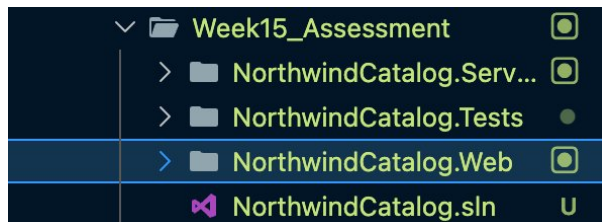
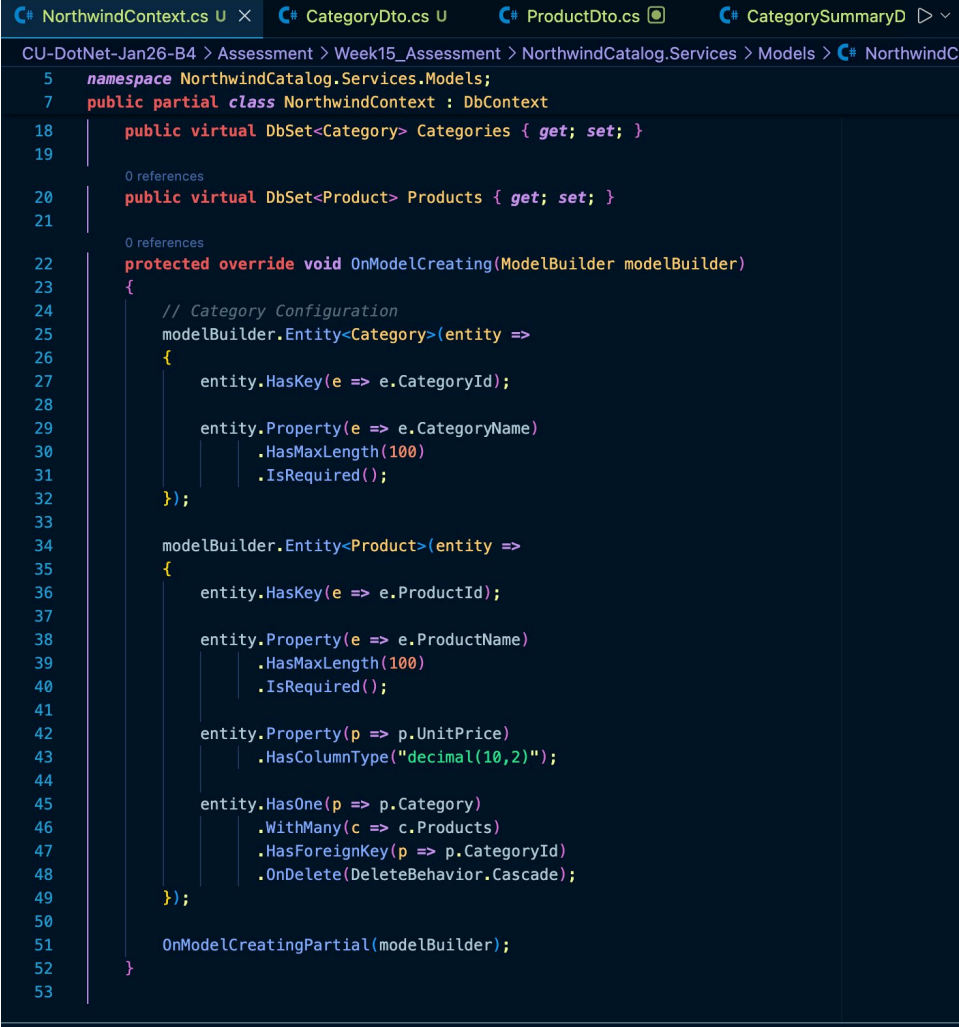


Figure 1 – Solution structure: NorthwindCatalog with Services, Web, and Tests projects

2. Configure Fluent API (OnModelCreating)

In NorthwindContext.cs, the OnModelCreating method was overridden to configure entity relationships and precision constraints using Fluent API. The Category entity was configured with a primary key on CategoryId and CategoryName set as required with a max length of 100 characters.

The Product entity was configured with a primary key on ProductId, ProductName as required (max 100 chars), UnitPrice stored as decimal(10,2), and a HasOne/WithMany relationship to Category with cascade delete on the foreign key CategoryId.



```
CU-DotNet-Jan26-B4 > Assessment > Week15_Assessment > NorthwindCatalog.Services > Models > NorthwindC
5 namespace NorthwindCatalog.Services.Models;
7 public partial class NorthwindContext : DbContext
18     public virtual DbSet<Category> Categories { get; set; }
19
20     public virtual DbSet<Product> Products { get; set; }
21
22     protected override void OnModelCreating(ModelBuilder modelBuilder)
23     {
24         // Category Configuration
25         modelBuilder.Entity<Category>(entity =>
26         {
27             entity.HasKey(e => e.CategoryId);
28
29             entity.Property(e => e.CategoryName)
30                 .HasMaxLength(100)
31                 .IsRequired();
32         });
33
34         modelBuilder.Entity<Product>(entity =>
35         {
36             entity.HasKey(e => e.ProductId);
37
38             entity.Property(e => e.ProductName)
39                 .HasMaxLength(100)
40                 .IsRequired();
41
42             entity.Property(p => p.UnitPrice)
43                 .HasColumnType("decimal(10,2)");
44
45             entity.HasOne(p => p.Category)
46                 .WithMany(c => c.Products)
47                 .HasForeignKey(p => p.CategoryId)
48                 .OnDelete(DeleteBehavior.Cascade);
49         });
50
51         OnModelCreatingPartial(modelBuilder);
52     }
53
```

Figure 2 – NorthwindContext.cs: OnModelCreating with Fluent API configuration

3. Data Transfer Objects (DTOs)

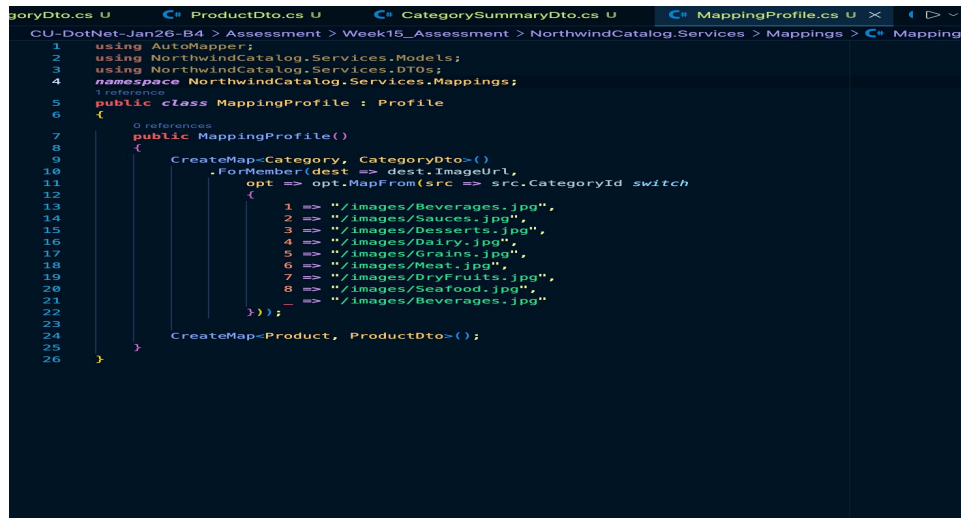
A DTOs folder was created inside the NorthwindCatalog.Services project. The following DTOs were defined:

- CategoryDto – maps CategoryId, CategoryName, and ImageUrl
- ProductDto – maps ProductId, ProductName, UnitPrice, UnitsInStock, and calculates InventoryValue (UnitPrice × UnitsInStock)
- CategorySummaryDto – provides a summary view across categories

The InventoryValue property in ProductDto is a computed property: UnitPrice multiplied by UnitsInStock, providing real-time inventory valuation per product.

4. AutoMapper Configuration (MappingProfile)

A MappingProfile.cs class was created inside a Mappings folder, inheriting from AutoMapper's Profile base class. The mapping from Category to CategoryDto uses a switch expression to assign the correct image URL based on CategoryId (1–8 mapped to Beverages, Sauces, Desserts, Dairy, Grains, Meat, DryFruits, Seafood). A default fallback maps to Beverages.jpg. Product to ProductDto uses a direct CreateMap with no custom member configuration required.



```
1 using AutoMapper;
2 using NorthwindCatalog.Services.Models;
3 using NorthwindCatalog.Services.DTOs;
4 namespace NorthwindCatalog.Services.Mappings;
5 public class MappingProfile : Profile
6 {
7     public MappingProfile()
8     {
9         CreateMap<Category, CategoryDto>()
10             .ForMember(dest => dest.ImageUrl,
11                 opt => opt.MapFrom(src => src.CategoryId switch
12                 {
13                     1 => "/images/Beverages.jpg",
14                     2 => "/images/Sauces.jpg",
15                     3 => "/images/Desserts.jpg",
16                     4 => "/images/Dairy.jpg",
17                     5 => "/images/Grains.jpg",
18                     6 => "/images/Meat.jpg",
19                     7 => "/images/DryFruits.jpg",
20                     8 => "/images/Seafood.jpg",
21                     _ => "/images/Beverages.jpg"
22                 }));
23         CreateMap<Product, ProductDto>();
24     }
25 }
26
```

Figure 3 – MappingProfile.cs: AutoMapper configuration for Category and Product DTOs

5. Repository Pattern Implementation

The repository pattern was implemented in the NorthwindCatalog.Services project inside a Repositories folder. The following files were created:

- ICategoryRepository – interface defining category data access contracts
- CategoryRepository – concrete implementation using NorthwindContext
- IProductRepository – interface defining product data access contracts
- ProductRepository – concrete implementation using LINQ projections

The GetCategorySummariesAsync method uses LINQ projections directly on IQueryable for efficient SQL generation, avoiding unnecessary data loading.

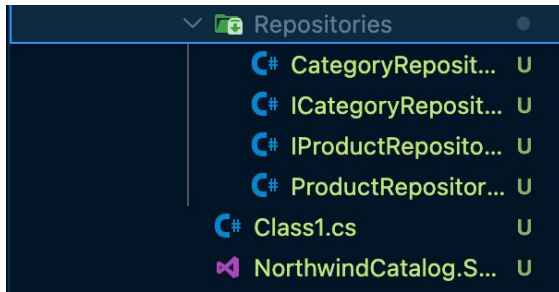
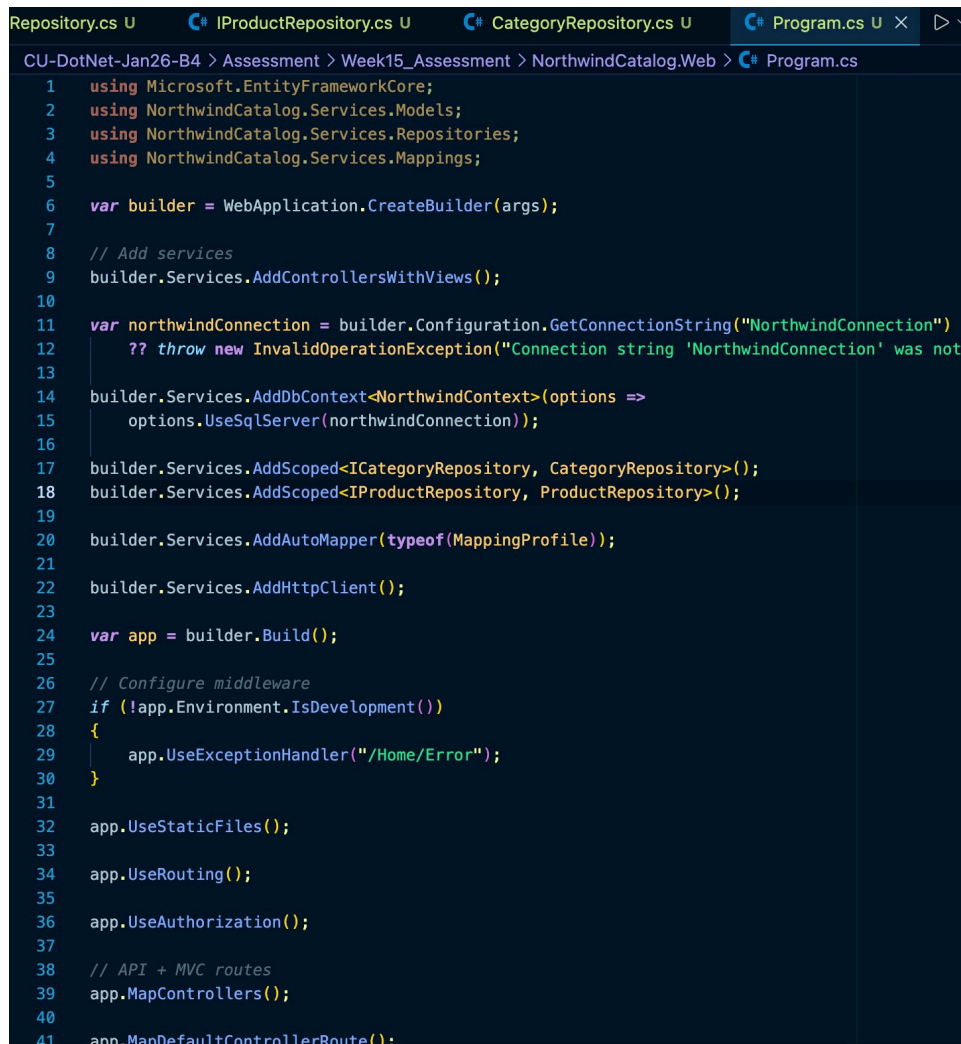


Figure 4 – Repositories folder showing interfaces and implementations

6. Dependency Injection & Program.cs Configuration

In **Program.cs** (**NorthwindCatalog.Web**), all services are registered using the built-in **ASP.NET Core DI** container. The connection string **NorthwindConnection** is retrieved from configuration with a null-guard that throws **InvalidOperationException** if missing. Key registrations include:

- `AddDbContext<NorthwindContext>` with `UseSqlServer`
- `AddScoped<ICategoryRepository, CategoryRepository>`
- `AddScoped<IProductRepository, ProductRepository>`
- `AddAutoMapper(typeof(MappingProfile))`
- `AddHttpClient()` for `IHttpClientFactory`



```
Repository.cs U  C# IProductRepository.cs U  C# CategoryRepository.cs U  C# Program.cs U X  ▾
CU-DotNet-Jan26-B4 > Assessment > Week15_Assessment > NorthwindCatalog.Web > C# Program.cs
1  using Microsoft.EntityFrameworkCore;
2  using NorthwindCatalog.Services.Models;
3  using NorthwindCatalog.Services.Repositories;
4  using NorthwindCatalog.Services.Mappings;
5
6  var builder = WebApplication.CreateBuilder(args);
7
8  // Add services
9  builder.Services.AddControllersWithViews();
10
11 var northwindConnection = builder.Configuration.GetConnectionString("NorthwindConnection")
12 ?? throw new InvalidOperationException("Connection string 'NorthwindConnection' was not
13
14 builder.Services.AddDbContext<NorthwindContext>(options =>
15     options.UseSqlServer(northwindConnection));
16
17 builder.Services.AddScoped<ICategoryRepository, CategoryRepository>();
18 builder.Services.AddScoped<IProductRepository, ProductRepository>();
19
20 builder.Services.AddAutoMapper(typeof(MappingProfile));
21
22 builder.Services.AddHttpClient();
23
24 var app = builder.Build();
25
26 // Configure middleware
27 if (!app.Environment.IsDevelopment())
28 {
29     app.UseExceptionHandler("/Home/Error");
30 }
31
32 app.UseStaticFiles();
33
34 app.UseRouting();
35
36 app.UseAuthorization();
37
38 // API + MVC routes
39 app.MapControllers();
40
41 app.MapDefaultControllerRoute();
```

Figure 5 – Program.cs: Service registrations, middleware pipeline, and routing

7. MVC Consumption & HttpClient

MVC Controllers (CategoriesController and ProductsController) were implemented in the NorthwindCatalog.Web project. Both controllers use `IHttpClientFactory` to consume the internal API endpoints created in the Services layer. `IMapper` is injected where additional DTO transformations are needed in the Web layer.

Endpoints consumed by the MVC layer include:

- GET api/categories – fetches all category DTOs for the Category Index view
- GET api/products/by-category/{id} – fetches products by category for the ByCategory view
- GET api/products/summary – fetches the category summary with inventory values

8. UI Implementation

The UI was developed using Bootstrap cards for the Categories Index page and a data table for the Products ByCategory page. The design uses a clean card grid layout displaying category images with a 'View Products' button for each category.

Category Index Page (Bootstrap Cards):

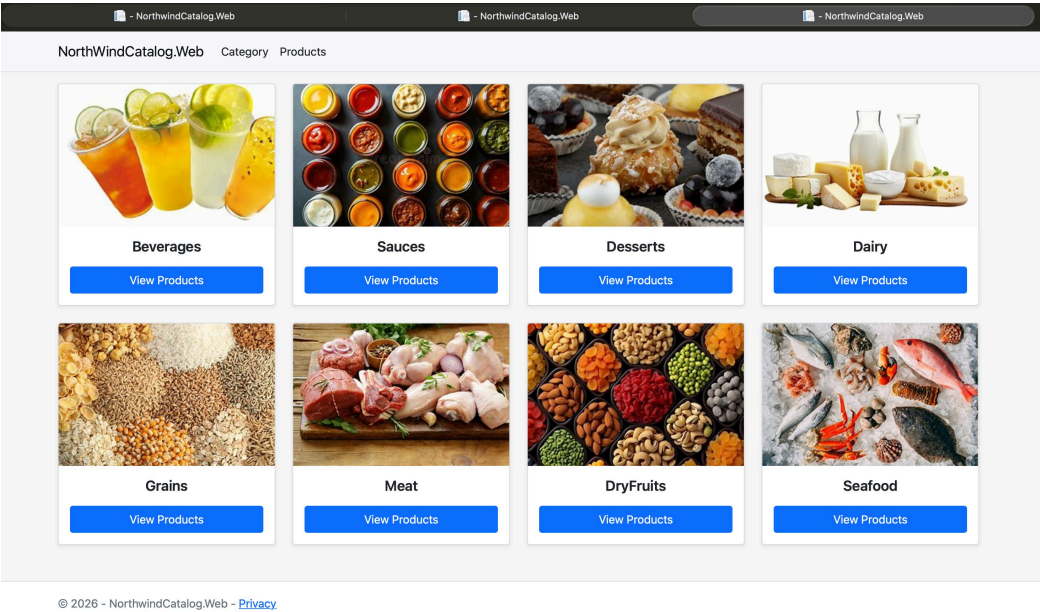


Figure 6 – Category Index: Bootstrap card grid displaying all 8 categories with images

Products ByCategory Page (Inventory Table):

NorthWindCatalog.Web Category Products			
ByCategory			
Create New			
ProductName	UnitPrice	UnitsInStock	InventoryValue
Chai	18.00	39	702.00
Updated Chang	19.00	17	323.00
Guaraná Fantástica	4.50	20	90.00
Sasquatch Ale	14.00	111	1554.00
Steeleye Stout	18.00	20	360.00
Côte de Blaye	263.50	17	4479.50
Chartreuse verte	18.00	69	1242.00
Ipoh Coffee	46.00	17	782.00
Laughing Lumberjack Lager	14.00	52	728.00
Outback Lager	15.00	15	225.00
Rhinbräu Klosterbier	7.75	125	968.75
Lakkaikööri	18.00	57	1026.00

Figure 7 – ByCategory Products table showing ProductName, UnitPrice, UnitsInStock, and InventoryValue

Summary

The NorthwindCatalog project successfully demonstrates a full-stack ASP.NET Core MVC application following clean architecture principles. The solution separates concerns across a Services class library (models, DTOs, repositories, mappings) and a Web project (controllers, views, DI). AutoMapper reduces boilerplate mapping code, the repository pattern abstracts data access, and Fluent API ensures precise database schema configuration. The UI provides an intuitive category browsing experience with real-time inventory value calculations.
